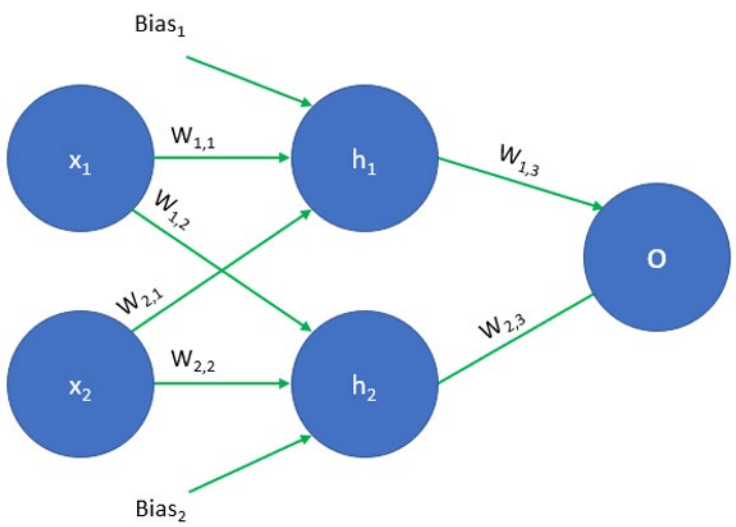
# Chapter 6:

# Artificial Neural Networks (ANN)

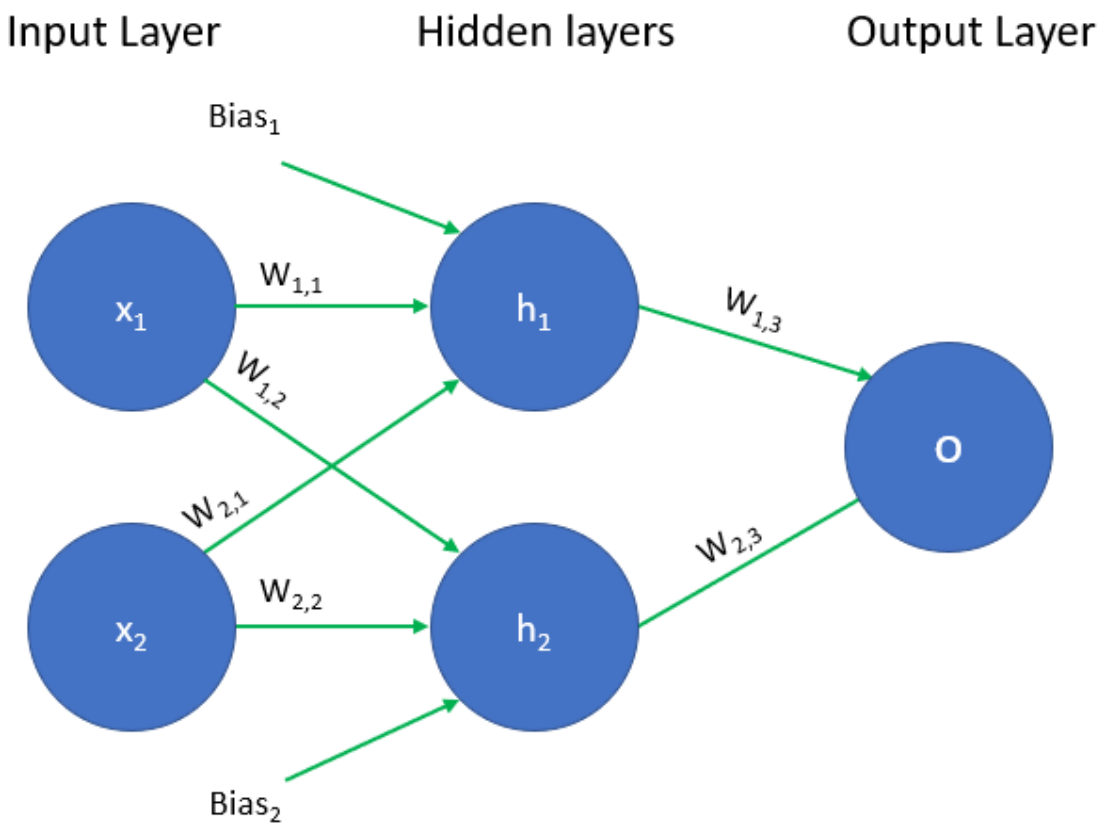
# Artificial Neural Networks (ANNs)

- ANNs are the algorithms inspired by the structure and functioning of the human brain.
- They are designed to recognize patterns and relationships in data, making them a fundamental component of deep learning systems.
- An artificial neural network is a computational model composed of layers of interconnected nodes, called neurons, that process and transmit information.
- Each connection between neurons has an associated weight, which determines the strength and influence of the connection.

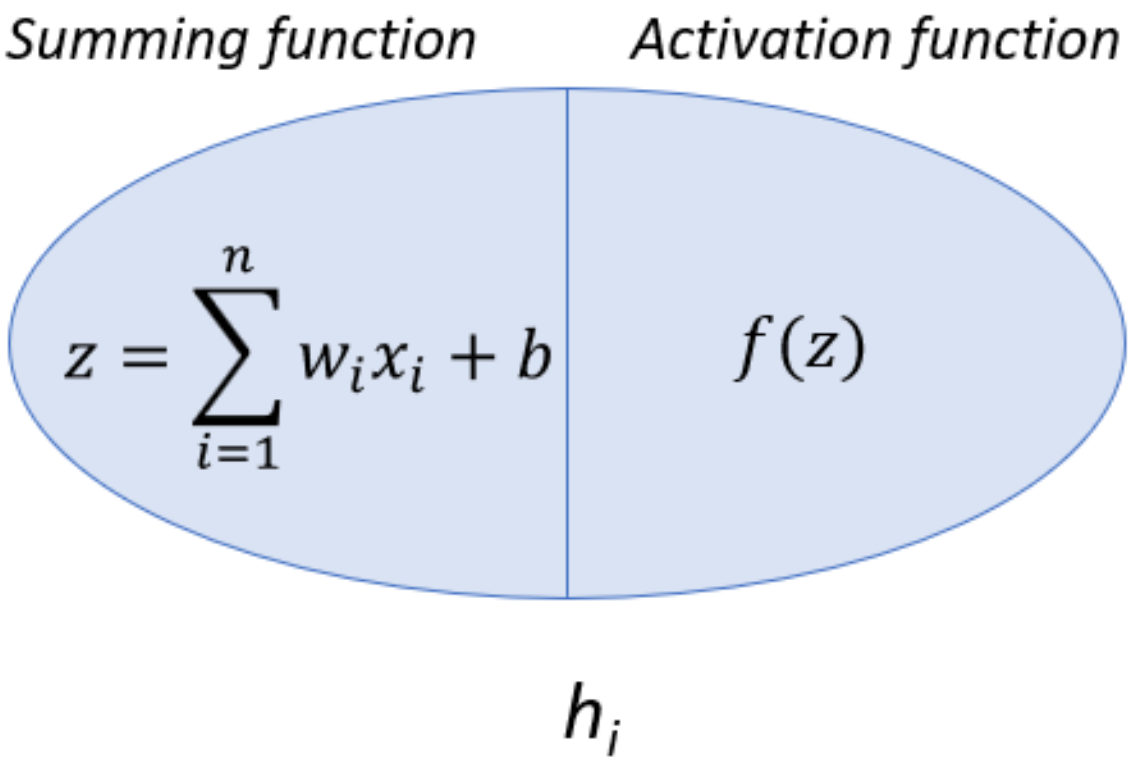


# Key Components of an ANN

**Input Layer**: Receives the input features for the model.
**Hidden Layers**: Perform computations and extract features from the input data.
**Output Layer**: Produces the final prediction or classification.
**Weights and Biases**: Parameters that the network adjusts during training to improve accuracy.

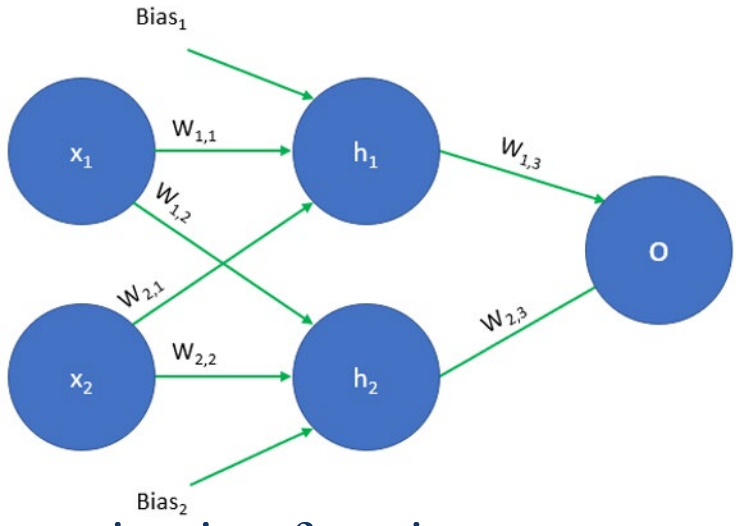


# Perceptron




**Activation Function**: Determines the output of a neuron and introduces non-linearity into the network.

# How Artificial Neural Networks Work




1. **Initialization**: Weights and biases are initialized randomly.

2. **Forward Propagation**:
   - Input data is passed through the network layer by layer.
   - Each neuron calculates a weighted sum of its inputs and applies an activation function.

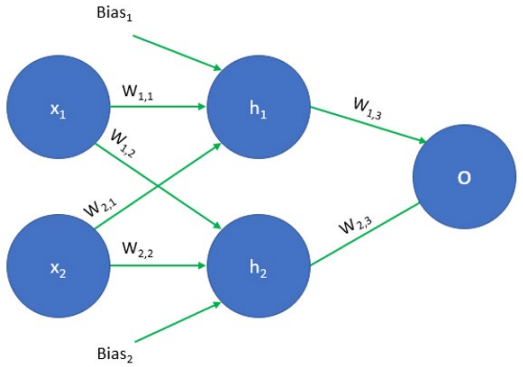
3. **Loss Calculation**: The difference between the predicted output and the actual target is computed using a loss function.

4. **Backpropagation**:
   - Gradients of the loss function with respect to weights are calculated using the chain rule.
   - Weights are updated using optimization algorithms like gradient descent.

5. **Iteration**: Steps 2–4 are repeated until the loss converges.

# Types of Artificial Neural Networks

- **Feedforward Neural Networks:** The simplest type, where data flows in one direction from input to output without cycles or loops.
- **Recurrent Neural Networks (RNNs):** Designed for sequential data, these networks have connections that form cycles, enabling them to maintain memory of previous inputs and model temporal dependencies.
- **Convolutional Neural Networks (CNNs):** Specialized for processing grid-like data such as images, using convolutional layers to automatically extract spatial features.
- **Autoencoders and Deep Belief Networks:** Used for unsupervised learning and dimensionality reduction.

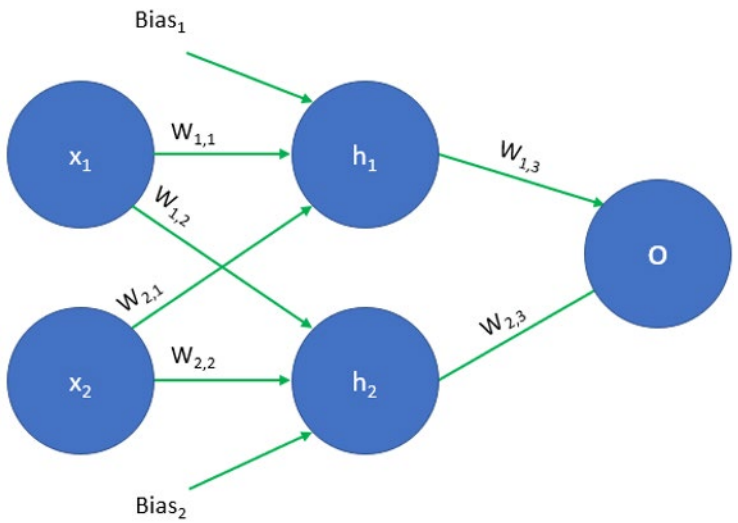


# Types of Artificial Neural Networks:

Feedforward Neural Networks (FNNs)

- FNNs are the **simplest** type of artificial neural networks, where information flows in **one direction**: from the input layer, through the hidden layers, to the output layer.

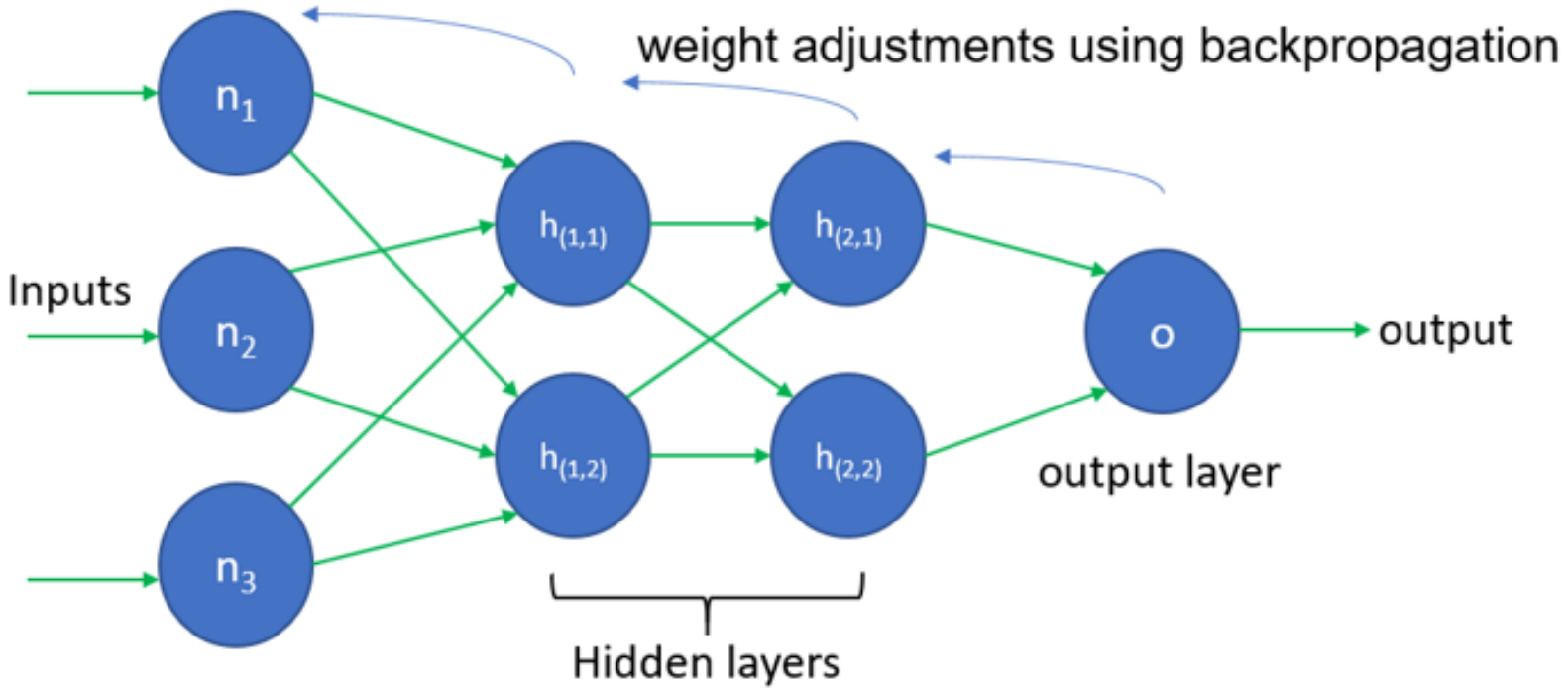
- This structure ensures there are **no cycles or loops** in the network, making FNNs the foundational architecture for many machine learning applications.



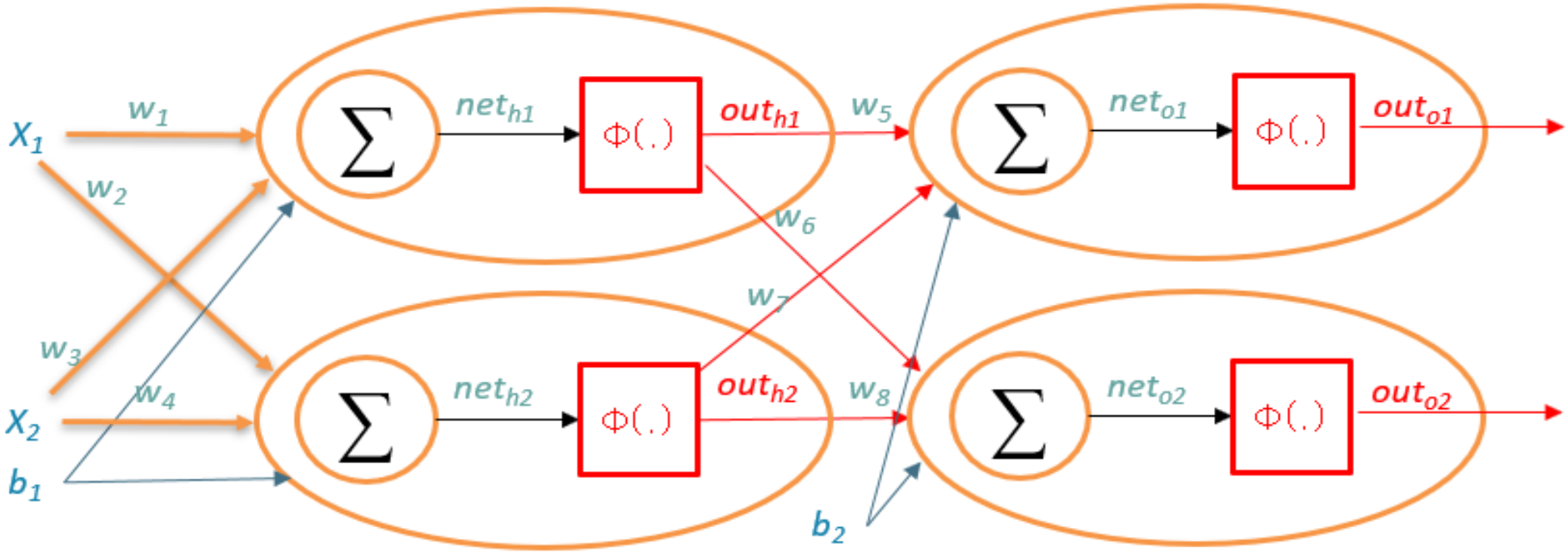
# Types of Artificial Neural Networks:

## Backpropagation

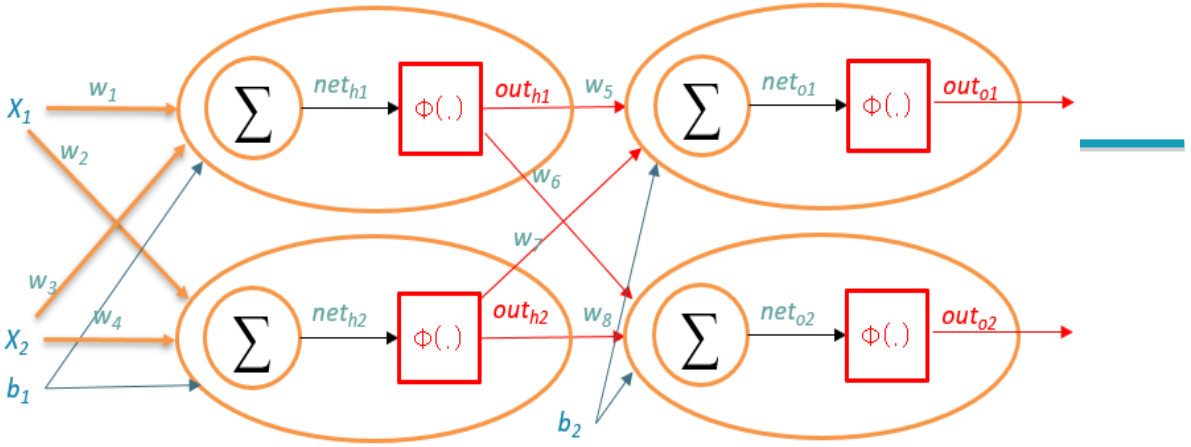
- Iteratively, it **modifies weights and biases** to minimize the cost function.
- The model adjusts these parameters in each epoch, minimizing loss by adhering to the error gradient.



# Mathematics



# Forward Pass Mathematics

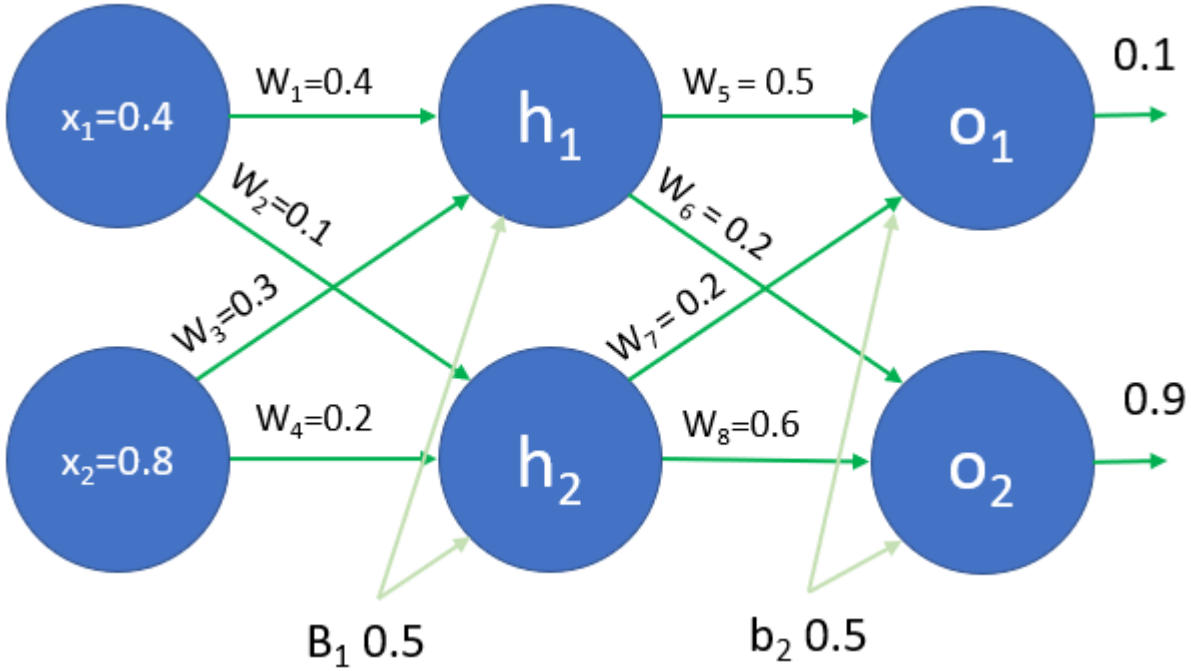



Chain rule of the calculus

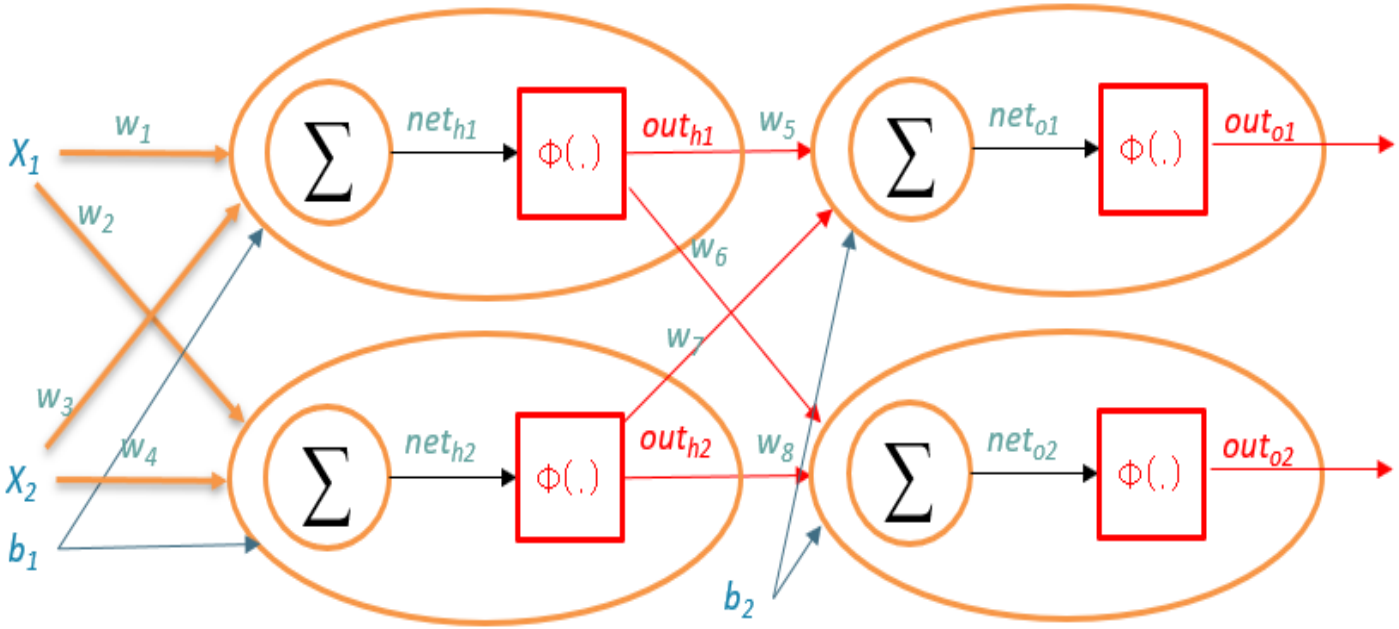
Leibniz's Notation:

$$\frac{d_y}{d_x} = \frac{d_y}{d_u} \cdot \frac{d_u}{d_x}$$

Example:



# Forward Pass Mathematics




Forward Pass Mathematics

$$net_{h_1} = w_1 * x1 + w_3 * x2 + b1$$

$= 0.4 \times 0.4 + 0.3 \times 0.8 + 0.5$

$= 0.9$

$$out_{h_1} = \frac{1}{1+e^{-net_{h_1}}}$$ ; Sigmoid activation function

$$= \frac{1}{1+e^{-0.9}}$$

$$= 0.35$$

$$net_{h_2} = w_2 * x_1 + w_4 * x_2 + b_1$$

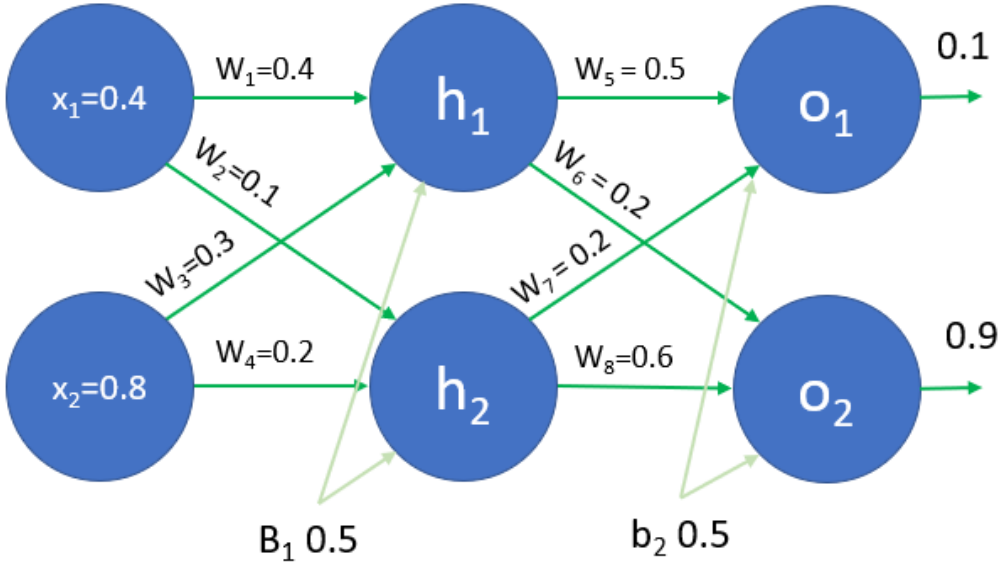
$$= 0.1 \times 0.4 + 0.2 \times 0.8 + 0.5$$

$$= 0.7$$

$$out_{h_2} = \frac{1}{1+e^{-net_{h_2}}}$$ ; Sigmoid activation function

$$= \frac{1}{1+e^{-0.7}}$$

$$= 0.33$$



# Forward Pass Mathematics

$$net_{o_1} = w_5 * out_{h_1} + w_7 * out_{h_2} + b_2$$

$$= 0.5 \times 0.35 + 0.2 \times 0.33 + 0.5$$

$$= 0.741$$

$$net_{o_2} = w6 * out_{h_1} + w_8 * out_{h_2} + b_2$$

$$= 0.2 \times 0.35 + 0.6 \times 0.33 + 0.5$$

$$= 1.398$$

$$out_{o_1} = \frac{1}{1 + e^{-net_{o_1}}}$$

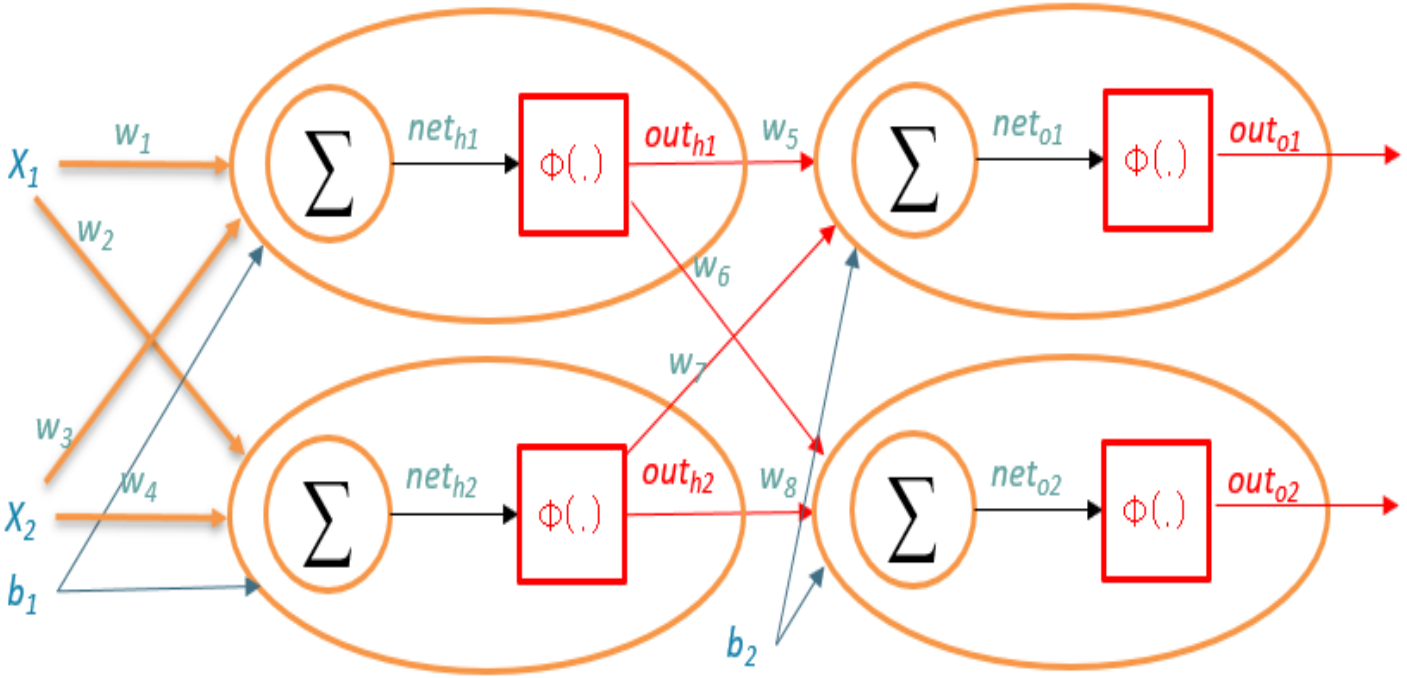
$$= \frac{1}{1+e^{-0.741}}$$

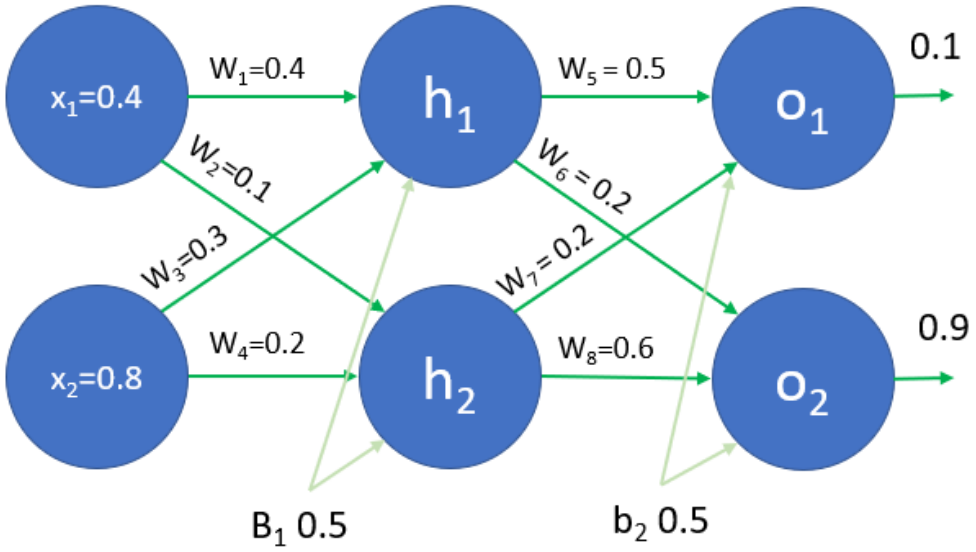
$$= 0.34$$

$$out_{o_2} = \frac{1}{1 + e^{-net_{o_2}}}$$

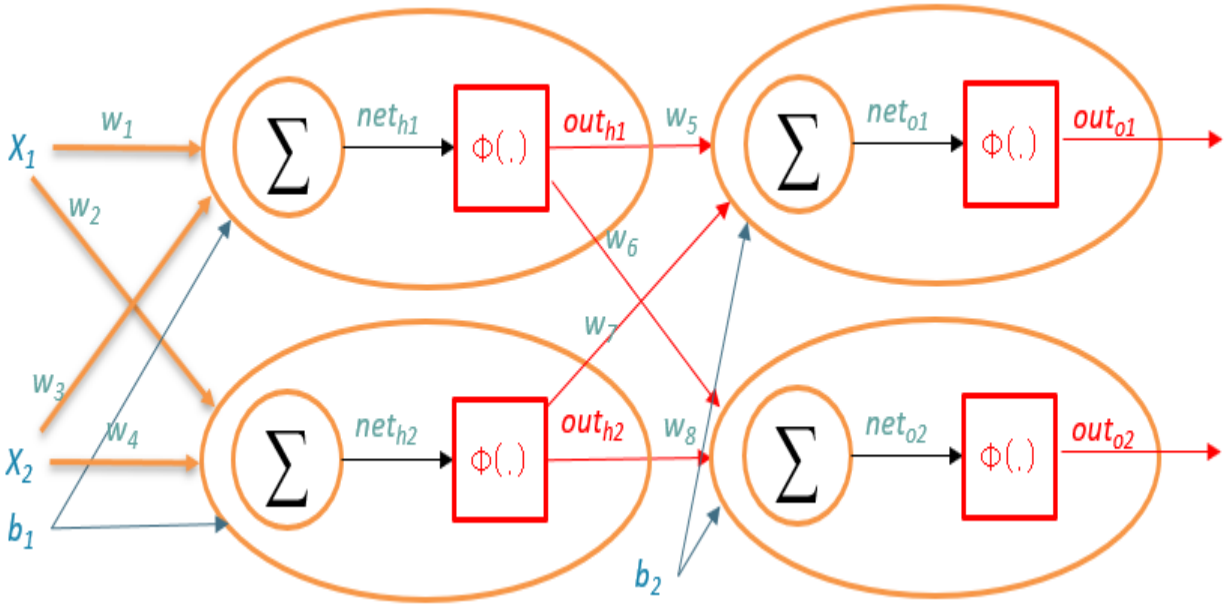
$$= \frac{1}{1+e^{-1.398}}$$

$$= 0.43$$



# Forward Pass Mathematics




***Error Totals***

$$E_{o_1} = \frac{1}{2}\left(target_{o_1} - output_{o_1}\right)^2$$

$$= \frac{1}{2}\,(0.1 - 0.34\,)^2$$

$$= 0.0288$$

$$E_{o_2} = \frac{1}{2}\left(target_{o_2} - output_{o_2}\right)^2$$

$$= \frac{1}{2}\,(0.9 - 0.43\,)^2$$

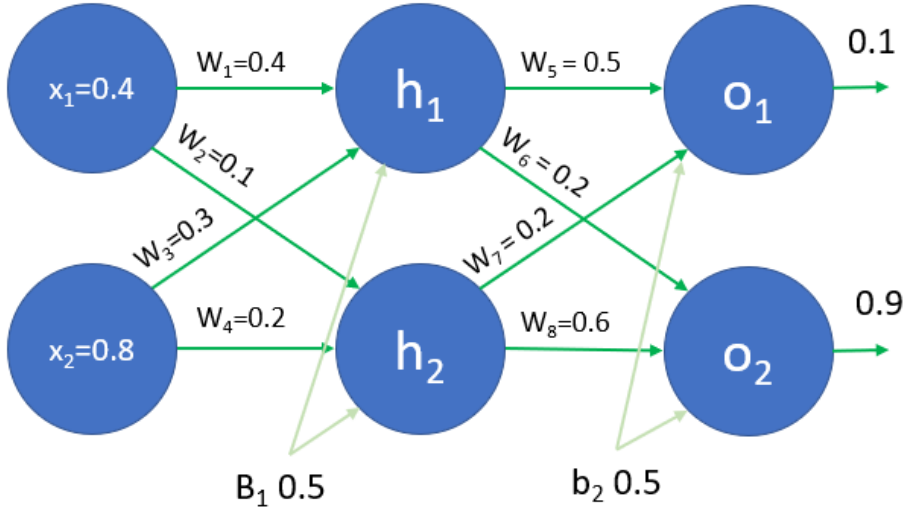
$$= 0.11$$

$$E_{total} = \sum \frac{1}{2}\,(target - output)^2$$

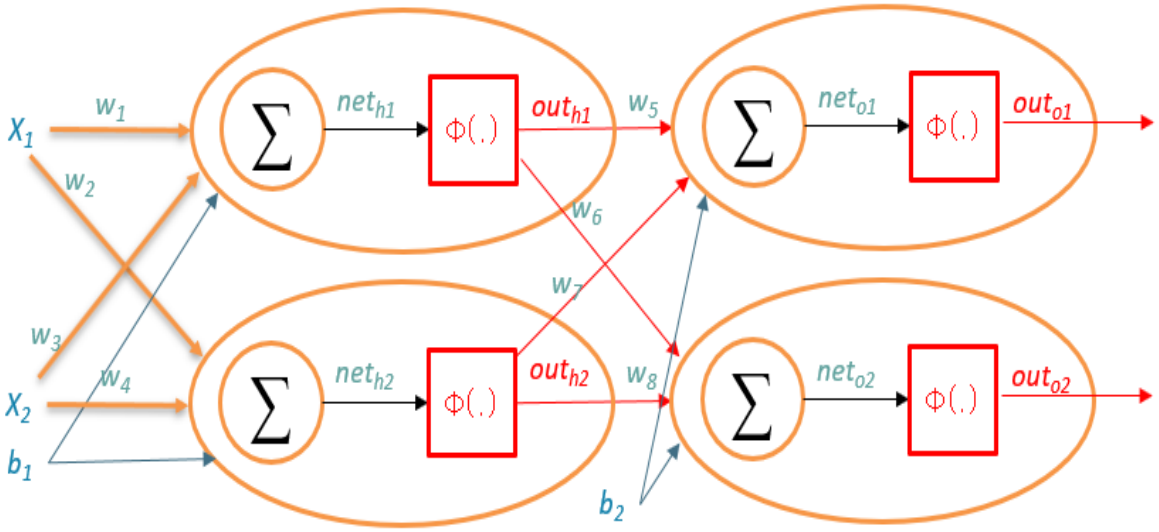
$$E_{total} = E_{o_1} + E_{o_2}$$

$$= 0.0288 + 0.11$$

$$= 0.1388$$



# Backward Pass Mathematics




**How much does the error in $w_5$ affect the $Error_{total}$?**

$$\frac{\partial Etotal}{\partial w_5} = ?$$

$$\frac{\partial net_{o_1}}{\partial w_5} * \frac{\partial out_{o_1}}{\partial net_{o_1}} * \frac{\partial E_{total}}{\partial out_{o_1}} = \frac{\partial E_{total}}{\partial w_5}$$

$$E_{total} = ½(target_{o1} - out_{o1})^2 + ½(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o_1}} = -2 * ½(target_{o1} - out_{o1})$$

$$= -(0.1 - 0.34)$$

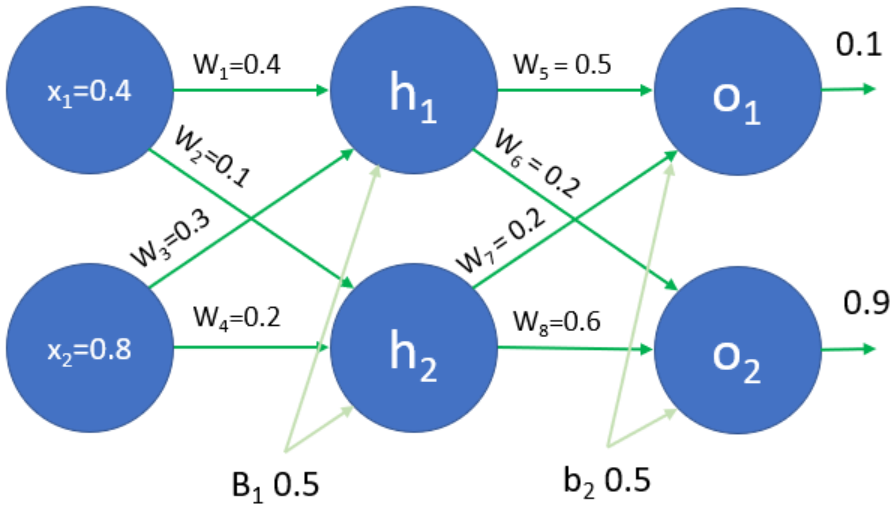
$$= 0.24$$

$$out_{o_1} = \frac{1}{1 + e^{-net_{o_1}}} \Big|$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = out_{o_1}(1 - out_{o_1})$$

$$= 0.34\ (1 - 0.34)$$

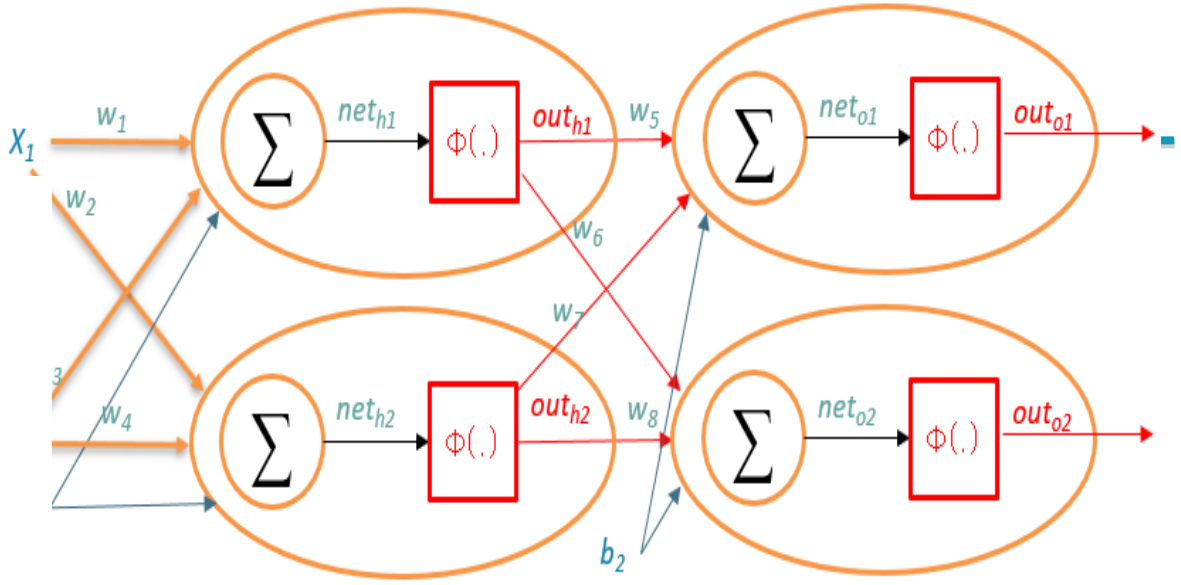
$$= 0.2244$$




$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

# Backward Pass Mathematics




$$net_{01} = w_5 * out_{h1} + w_7 * out_{h2} + b_2$$

$$\frac{\partial net_{o_1}}{\partial w_5} = out_{h_1}$$

$$= 0.35$$

$$\frac{\partial net_{o_1}}{\partial w_5} * \frac{\partial out_{o_1}}{\partial net_{o_1}} * \frac{\partial E_{total}}{\partial out_{o_1}} = \frac{\partial E_{total}}{\partial w_5}$$

$$= 0.35 * 0.2244 * 0.24$$

$$= 0.00188496$$

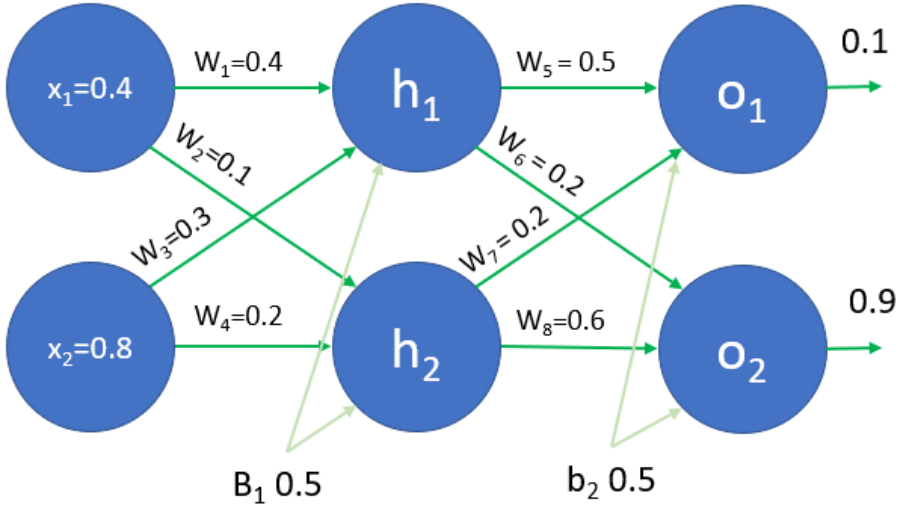
$$w_5^+ = w_5 - \alpha * \frac{\partial Etotal}{\partial w_5} \qquad \alpha : learning\ rate$$

If we consider $\alpha = 0.5$, $w_5^+ = 0.5 - 0.5 * 0.00188496 = 0.49905752$

$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

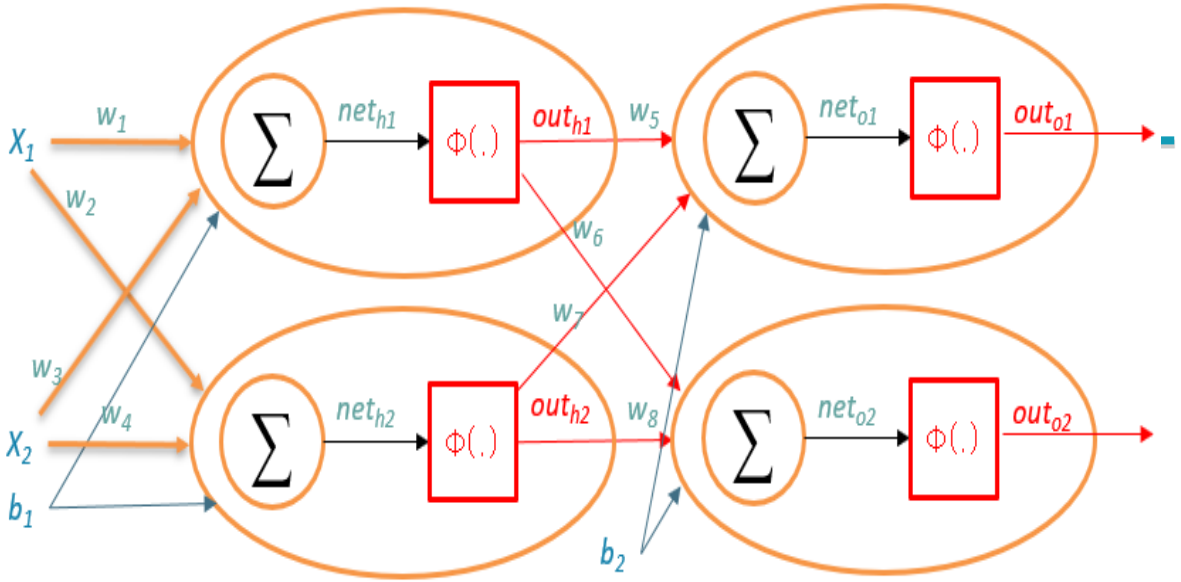
$$\frac{\partial net_{o_1}}{\partial w_5} = 0.35$$



# Backward Pass Mathematics

**Finally, how much does the error in $w_1$ affect the $Error_{total}$?**

$$\underbrace{\frac{\partial neth_1}{\partial w_1}}_{C} * \underbrace{\frac{\partial outh_1}{\partial neth_1}}_{B} * \underbrace{\frac{\partial Etotal}{\partial outh_1}}_{A} = \frac{\partial Etotal}{\partial w_1}$$




A

$$\frac{\partial Etotal}{\partial outh_1} = \frac{\partial Eo_1}{\partial outh_1} + \frac{\partial Eo_2}{\partial outh_1}$$

$$\frac{\partial Eo_1}{\partial outh_1} = \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial net_{o_1}} * \frac{\partial net_{o_1}}{\partial out_{h_1}}$$

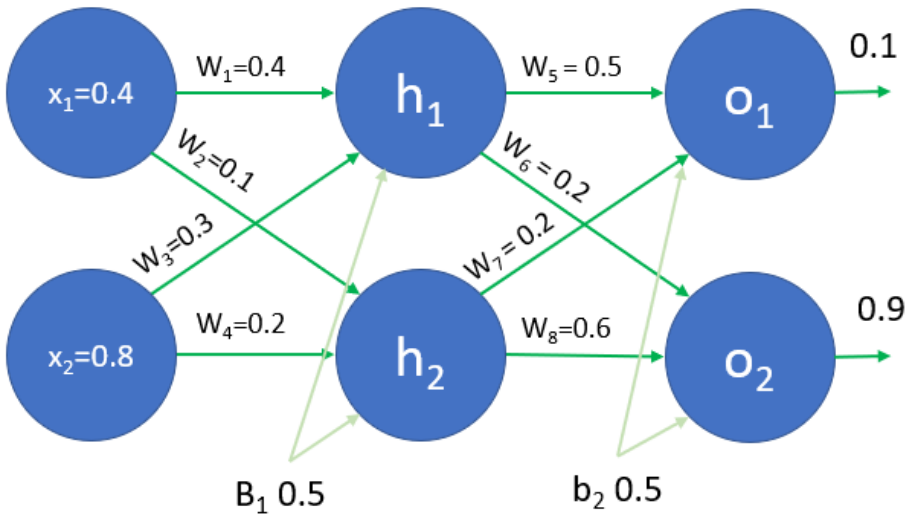
$$\frac{\partial E_{o_1}}{\partial out_{o_1}} = 0.24, \quad \frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

$$net_{o_1} = w_5 * out_{h_1} + w_7 * out_{h_2} + b_2$$

$$\frac{\partial neto_1}{\partial outh_1} = W_5 = 0.5$$

$$\frac{\partial Eo_1}{\partial outh_1} = 0.24 * 0.2244 * 0.5$$

$$= 0.026928$$




$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

$$\frac{\partial net_{o_1}}{\partial w_5} = 0.35$$

$$\frac{\partial Eo_1}{\partial outh_1} = 0.026928$$

# Backward Pass Mathematics

**Finally, how much does the error in $w_1$ affect the $Error_{total}$?**

$$\underbrace{\frac{\partial neth_1}{\partial w_1}}_{C} * \underbrace{\frac{\partial outh_1}{\partial neth_1}}_{B} * \underbrace{\frac{\partial Etotal}{\partial outh_1}}_{A} = \frac{\partial Etotal}{\partial w_1}$$

$$\frac{\partial Etotal}{\partial outh_1} = \frac{\partial Eo_1}{\partial outh_1} + \frac{\partial Eo_2}{\partial outh_1}$$

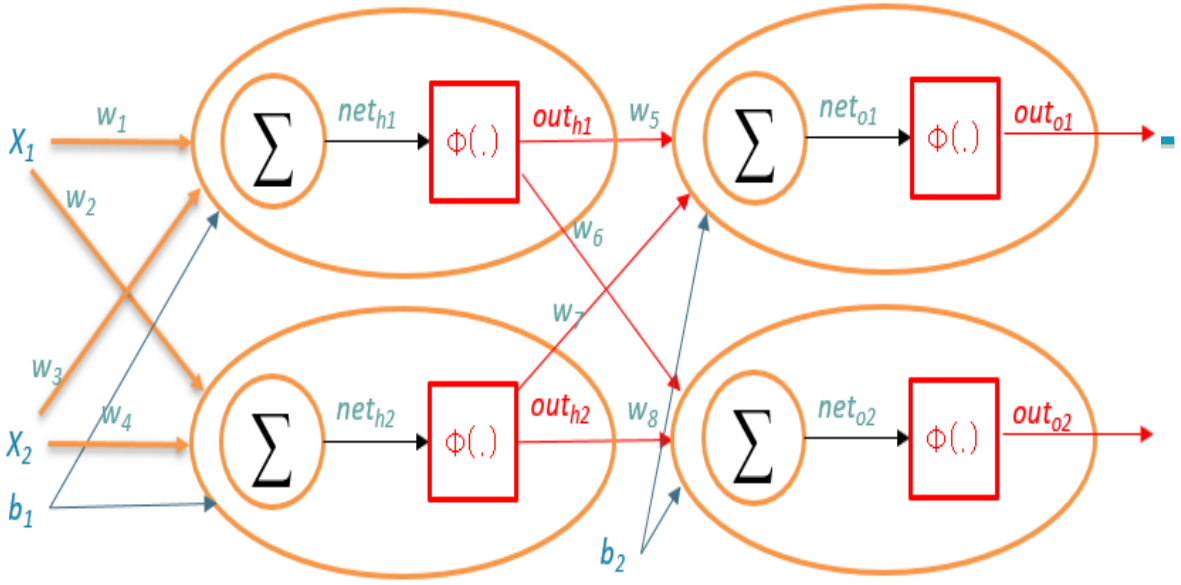
$$\frac{\partial Etotal}{\partial outh_1} = 0.026928 + \frac{\partial Eo_2}{\partial outh_1}$$

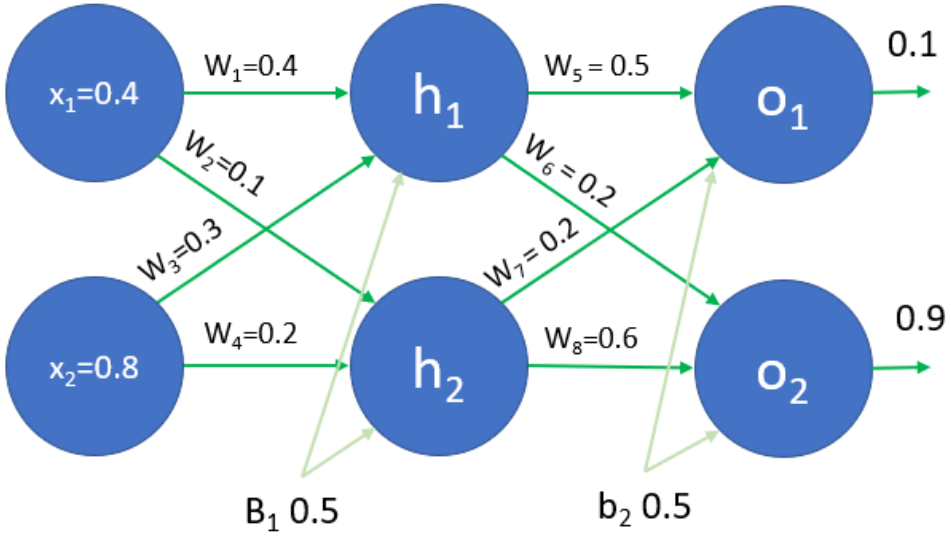
$$\frac{\partial E_{o_2}}{\partial out_{h_1}} = \frac{\partial E_{o_2}}{\partial out_{o_2}} * \frac{\partial out_{o_2}}{\partial net_{o_2}} * \frac{\partial net_{o_2}}{\partial out_{h_1}}$$

$$\frac{\partial E_{o_2}}{\partial out_{o_2}} = -2 * \frac{1}{2}(target_{o_2} - out_{o_2})$$

= -(0.9-0.43)

=-0.47



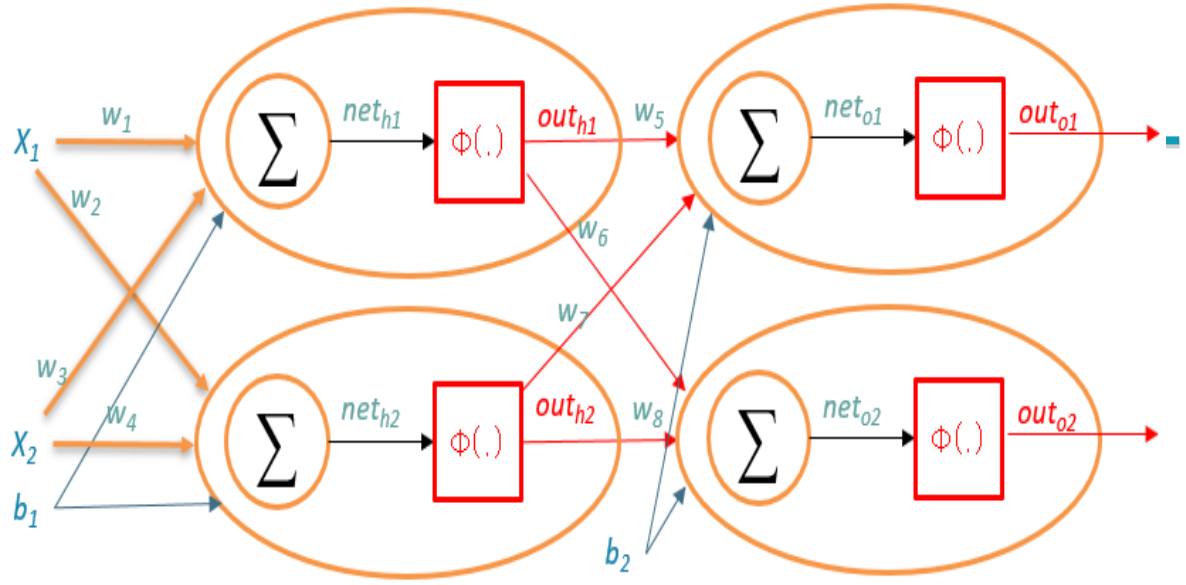

$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

$$\frac{\partial net_{o_1}}{\partial w_5} = 0.35$$

$$\frac{\partial Eo_1}{\partial outh_1} = 0.026928$$

# Backward Pass Mathematics




$$\frac{\partial E_{o_2}}{\partial out_{h_1}} = \frac{\partial E_{o_2}}{\partial out_{o_2}} * \frac{\partial out_{o_2}}{\partial net_{o_2}} * \frac{\partial net_{o_2}}{\partial out_{h_1}}$$

$$\frac{\partial E_{o_2}}{\partial out_{o_2}} = -2 * \frac{1}{2}(target_{o_2} - out_{o_2})$$

= -(0.9-0.43)

=-0.47

$$\frac{\partial out_{o_2}}{\partial net_{o_2}} = out_{o_2}(1 - out_{o_2})$$

=0.43(1-0.43)= 0.2451

$$\frac{\partial net_{o_2}}{\partial out_{h_1}} = w_6 * out_{h_1} + w_8 * out_{h_2} + b_2$$

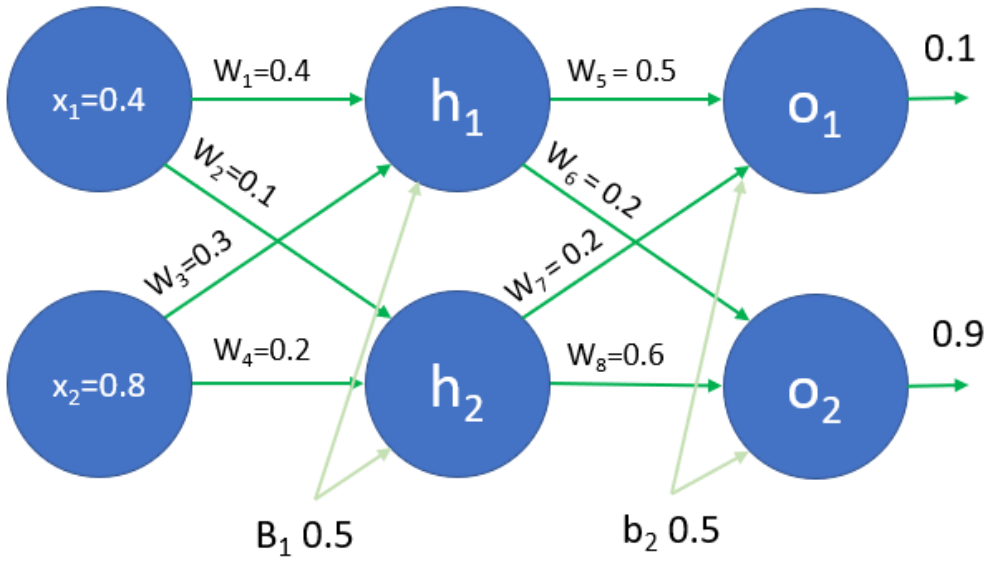
$$= w_6 = 0.3$$

$$\frac{\partial E_{o_2}}{\partial out_{h_1}} = -0.47 * 0.2451 * 0.3$$

= -0.0345591

$$\frac{\partial Etotal}{\partial outh_1} = 0.026928 + -0.0345591$$

= -0.0076311




$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

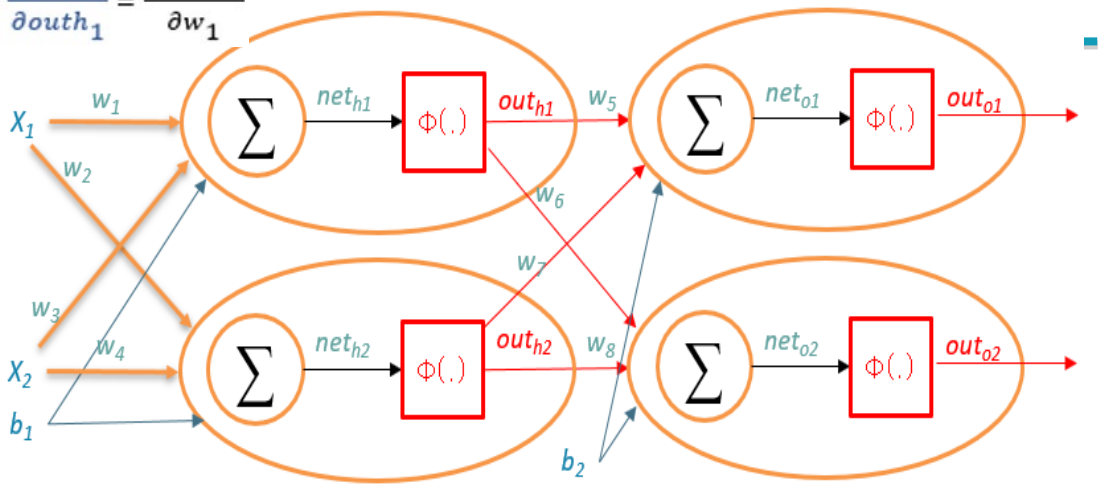
$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

$$\frac{\partial net_{o_1}}{\partial w_5} = 0.35$$

$$\frac{\partial Eo_1}{\partial outh_1} = 0.026928$$

# Backward Pass Mathematics

$$\underbrace{\frac{\partial neth_1}{\partial w_1}}_{C} * \underbrace{\frac{\partial outh_1}{\partial neth_1}}_{B} * \underbrace{\frac{\partial Etotal}{\partial outh_1}}_{A} = \frac{\partial Etotal}{\partial w_1}$$




B

$$out_{h1} = \frac{1}{1 + e^{-neth1}}$$

$$\frac{\partial outh_1}{\partial neth_1} = out_{h_1}(1 - out_{h_1})$$

$$= 0.35\ (1 - 0{,}35)$$

$$= 0.2275$$

C

$$net_{h1} = w_1 * x_1 + w_3 * x_2 + b_1$$

$$\frac{\partial neth_1}{\partial w_1} = x_1 = 0.4$$

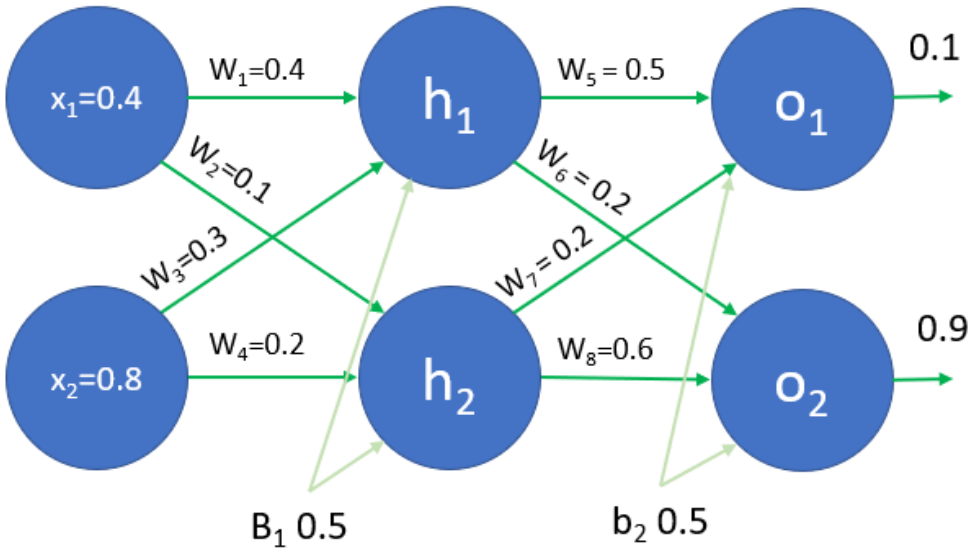
$$\frac{\partial E_{total}}{\partial w_1} = -0.0076311 * 0.2275 * 0.4$$

= -0.0006944301

$$w^+_1 = w_1 - \alpha * \frac{\partial Etotal}{\partial w_1}$$

= 0.4 – 0.5 * (- 0.0006944301)

= 0.40034721505




$$\frac{\partial E_{total}}{\partial out_{o_1}} = 0.24$$

$$\frac{\partial out_{o_1}}{\partial net_{o_1}} = 0.2244$$

$$\frac{\partial net_{o_1}}{\partial w_5} = 0.35$$

$$\frac{\partial Eo_1}{\partial outh_1} = 0.026928$$

# Advantages of Backpropagation for Neural Network Training

- Efficiency: Backpropagation, particularly in deep networks, speeds up learning by immediately changing weights depending on mistake.
- Generalization: It improves prediction accuracy on unseen cases by assisting models in making good generalizations to fresh data.

# Convolutional Neural Networks (CNNs)

- CNNs are designed for processing structured data like images.

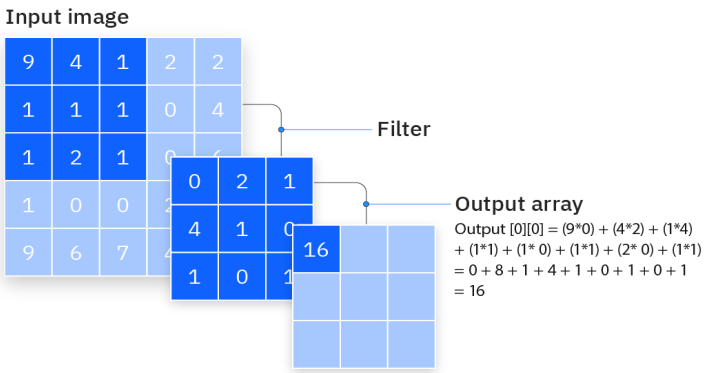
- By leveraging convolutional layers and filters, CNNs can automatically and efficiently learn spatial hierarchies of features from input data.

**Architecture of Convolutional Neural Networks:**

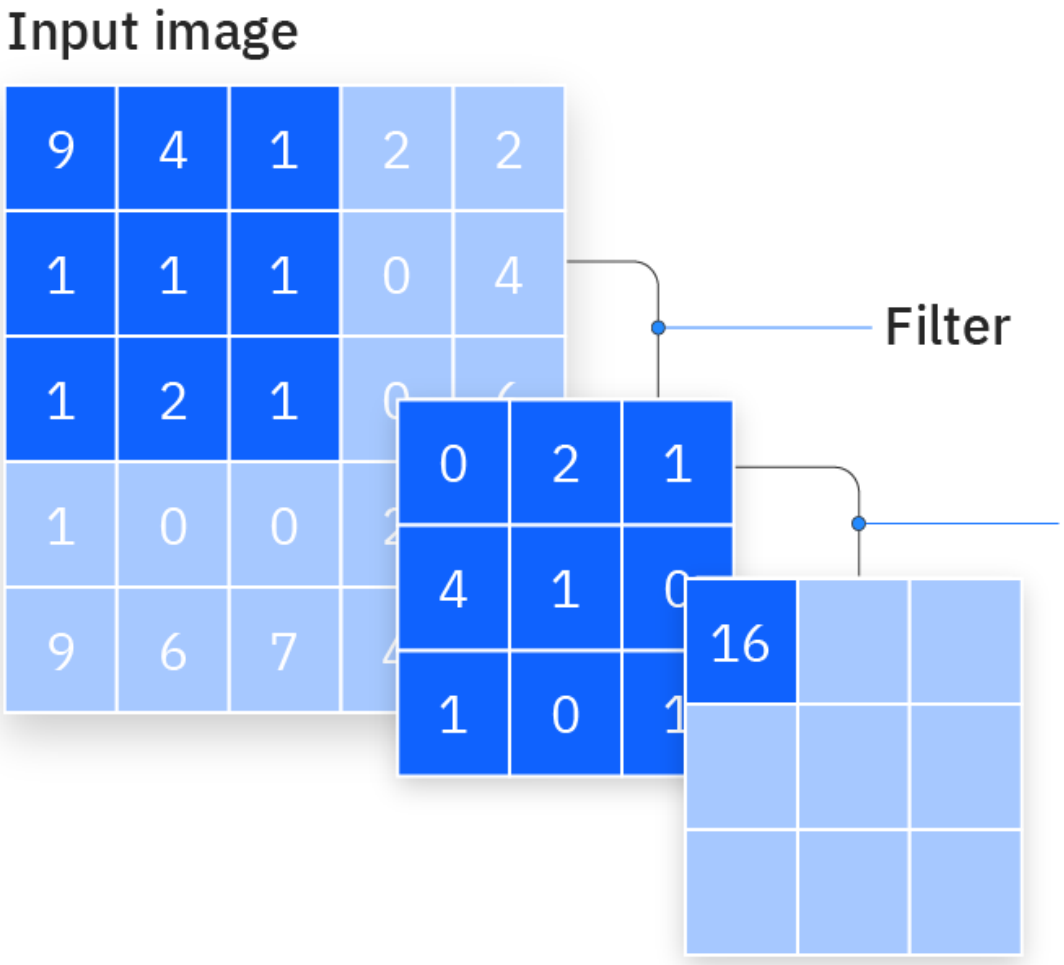
1. Input Layer: Accepts raw pixel values.
2. Convolutional + Activation Layers: Extract features.
3. Pooling Layers: Down-sample features.
4. Fully Connected Layers: Output predictions.

# Convolutional Layer

- This layer requires several key components to function effectively:
  - input data,
    - E.g.: Color image
  - a filter (also known as a kernel or feature detector), and
    - E.g.: often 3x3 matrix.
    - It moves across the image, it computes a dot product between the input pixels and the filter weights at each position
  - a feature map
    - This dot product is then recorded in an output array, which ultimately forms the feature map



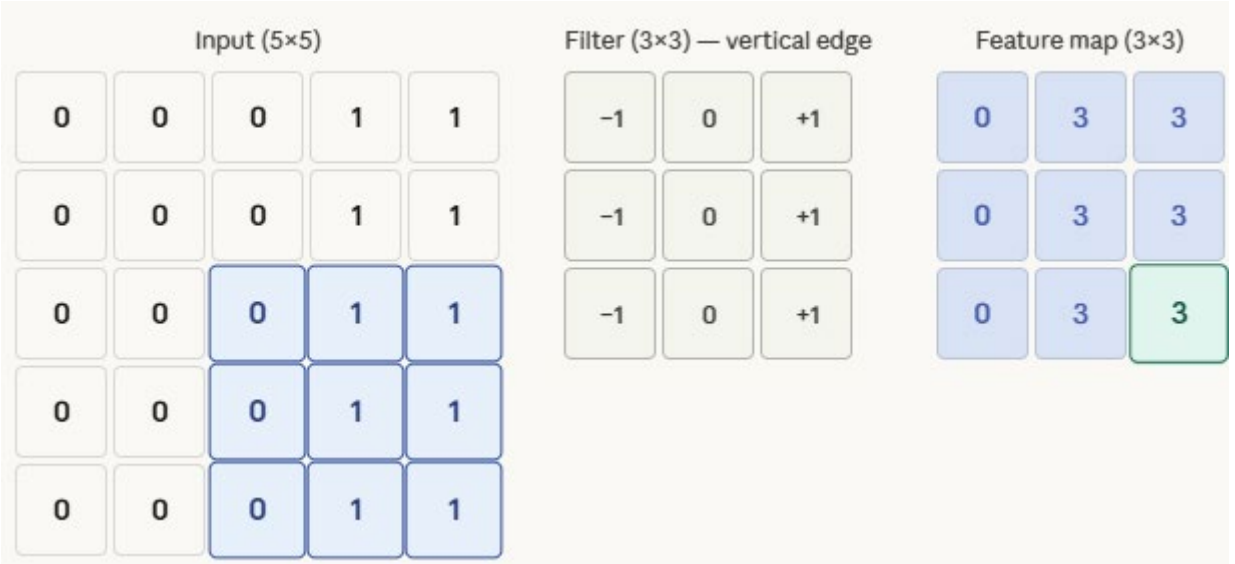
# Convolutional Layer

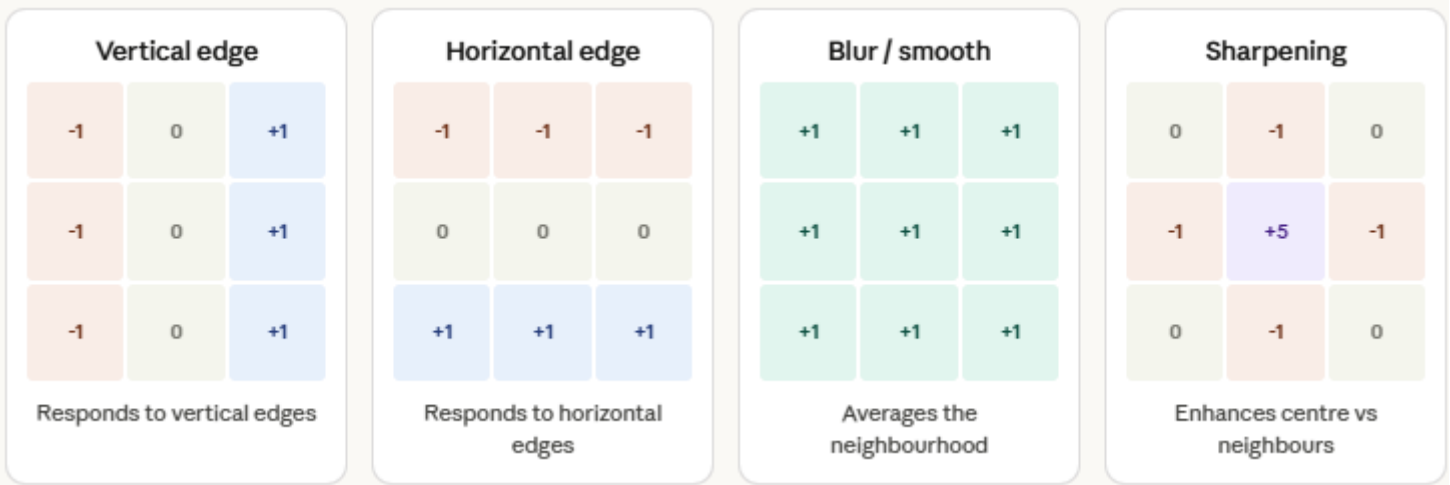



Output [0][0] = (9*0) + (4*2) + (1*4)
+ (1*1) + (1* 0) + (1*1) + (2* 0) + (1*1)
= 0 + 8 + 1 + 4 + 1 + 0 + 1 + 0 + 1
= 16

# Convolutional Layer

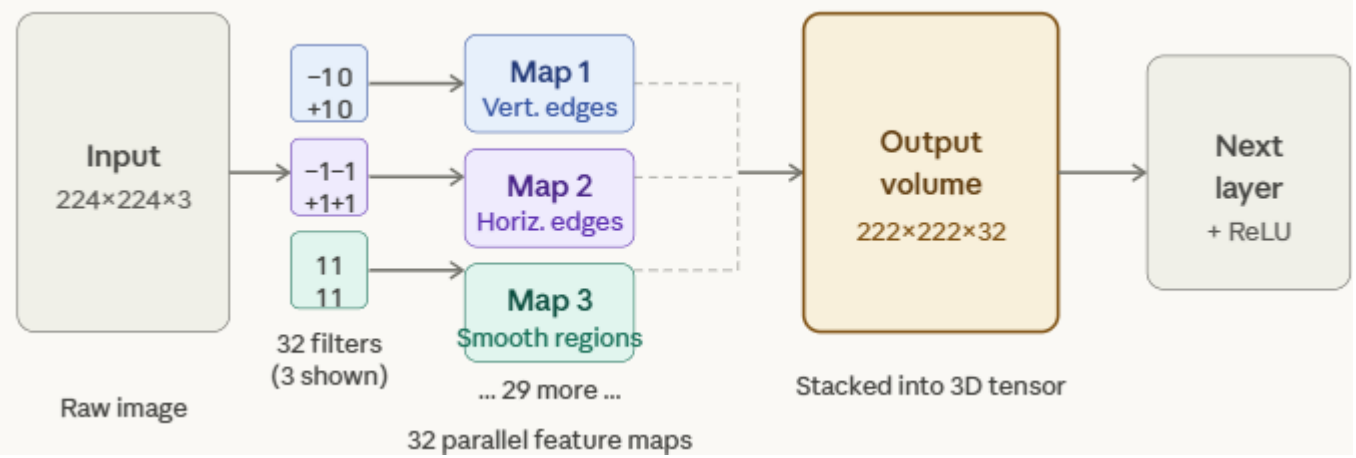
Each filter is trained to respond strongly to one type of pattern: a vertical edge, a diagonal, a curve, a texture.

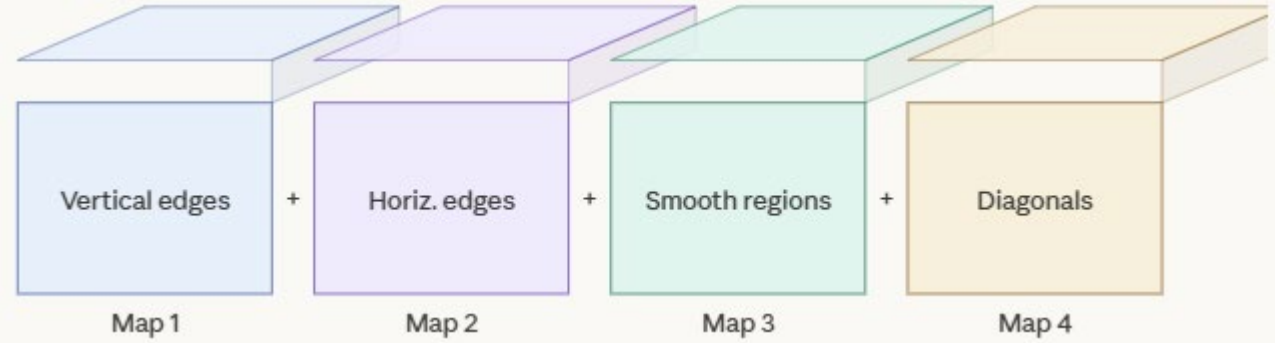


# Convolutional Layer

One filter produces one feature map. A real conv layer applies many filters at once, producing a "stack" of feature maps — one per filter. Each map highlights a different aspect of the input.




So one filter in layer 2 is simultaneously asking: "at this location, are there vertical edges AND horizontal curves AND smooth regions AND ... (all 32 channels combined)?"

# Pooling Layer

- This process involves reducing the spatial dimensions of the input data, thereby decreasing the number of parameters and enhancing computational efficiency.

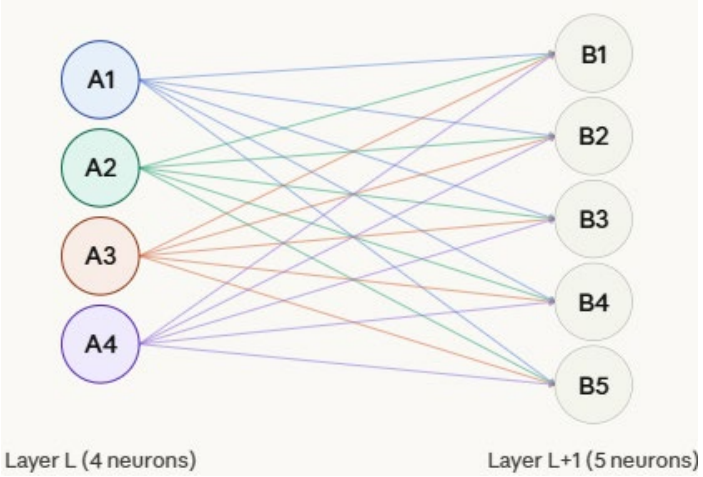
- Similar to the convolutional layer, the pooling operation involves sweeping a filter across the entire input. However, unlike convolutional filters, pooling filters do not have weights; instead, they apply an **aggregation function** to the values within the receptive field to populate the output array.

# Fully-connected Layer

- The fully connected (FC) layer — also called a dense layer — is typically the final stage of a CNN. After the convolutional and pooling layers have extracted and compressed spatial features from the image, the FC layer interprets those features and produces the final classification or regression output.

- The name comes from its structure: every neuron in the layer is connected to every neuron in the previous layer — no spatial locality, no weight sharing, just a complete web of connections.

- Fully connected layers typically employ the softmax activation function for classification purposes.



# Calculation Example:

Assume we have the following:

- Input image: 5x5 matrix
- Filter (kernel): 3x3 matrix
- Stride: 1
- Padding: 0 (valid padding)

1. **Input Image**:

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 0 | 1 |
| 0 | 1 | 2 | 1 | 2 |
| 3 | 1 | 0 | 2 | 3 |
| 0 | 0 | 1 | 3 | 1 |
| 2 | 1 | 3 | 1 | 0 |

2. **Filter (Kernel)**:

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

**3. Convolution Operation**:
For each position of the filter on the input image, calculate the element-wise product and sum the results.

**First Position (1,1):**

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | | 1 | 0 | 1 |
| 0 | 1 | 2 | * | 0 | 1 | 0 |
| 3 | 1 | 0 | | 1 | 0 | 1 |

Sum of products: 1*1 + 2*0 + 3*1 + 0*0 + 1*1 + 2*0 + 3*1 + 1*0 + 0*1 = 1 + 0 + 3 + 0 + 1 + 0 + 3 + 0 + 0 = 8

**Second Position (1,2):**

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | 3 | 0 | | 1 | 0 | 1 |
| 1 | 2 | 1 | * | 0 | 1 | 0 |
| 1 | 0 | 2 | | 1 | 0 | 1 |

Sum of products: 2*1 + 3*0 + 0*1 + 1*0 + 2*1 + 1*0 + 1*1 + 0*0 + 2*1 = 2 + 0 + 0 + 0 + 2 + 0 + 1 + 0 + 2 = 7

By continuing the convolution operation across the entire image, we will get the resulting feature map.

| | | |
|---|---|---|
| 8 | 7 | 6 |
| 5 | 8 | 7 |
| 4 | 5 | 6 |

# Types of Convolutional Neural Network

- AlexNet
- VGGNet
- GoogLeNet
- ResNet
- ZFNet

# Activation Functions

AFs enable Neural Networks to learn complex patterns and make decisions based on input data.

**Importance of Activation Functions**

1. **Non-Linearity**: Without activation functions, neural networks would behave like linear regression models, regardless of how many layers they have. This limitation would prevent them from capturing complex relationships in the data.
2. **Gradient Propagation**: Activation functions provide gradients necessary for the backpropagation algorithm, which is essential for updating weights during training.
3. **Decision Making**: They help determine whether a neuron should be activated based on the weighted sum of its inputs, thus influencing the network's output

# Types of Activation Functions

## 1. Linear Activation Function

- **Definition**: The output is directly proportional to the input.
- **Mathematical Representation**: $f(x) = x$
- **Limitations**: It does not allow for backpropagation effectively and collapses multiple layers into a single layer, limiting the model's complexity.

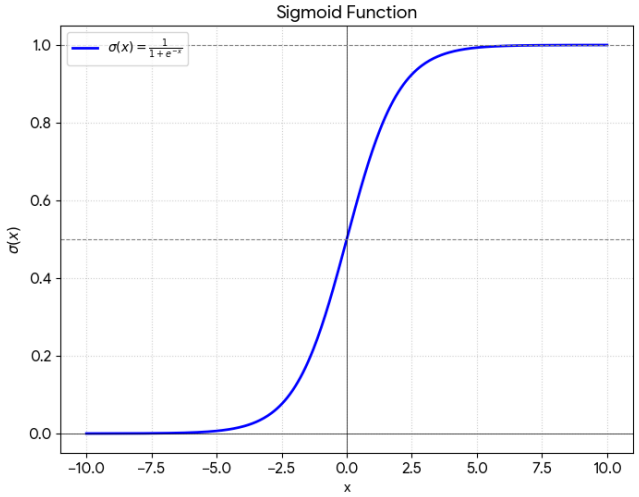
## 2. Binary Step Function

- **Definition**: Activates a neuron based on a threshold.
- **Mathematical Representation**:

$$f(x) = \begin{cases} 1 \; if \; x > threshold \\ 0 \; otherwise \end{cases}$$

- **Limitations**: It cannot handle multi-class outputs and has a zero gradient for non-active neurons, hindering backpropagation.

# Types of Activation Functions

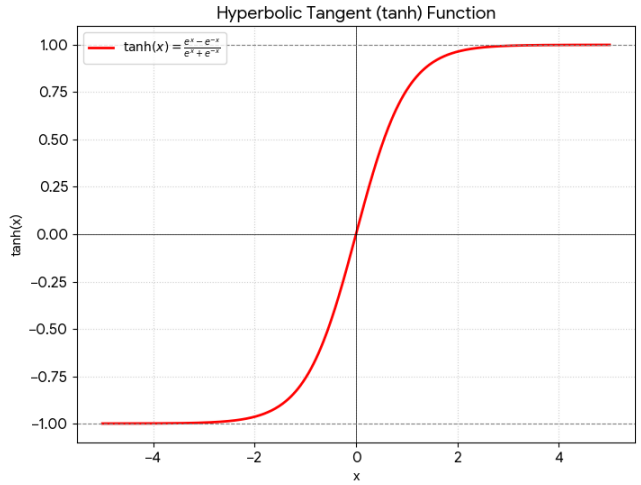



## 3. Sigmoid Function

- **Definition**: Maps any input to a value between 0 and 1.
- **Mathematical Representation**:

$$f(x) = \frac{1}{1+e^{-x}}$$

- **Limitations**: Prone to vanishing gradient problems for very high or low input values.



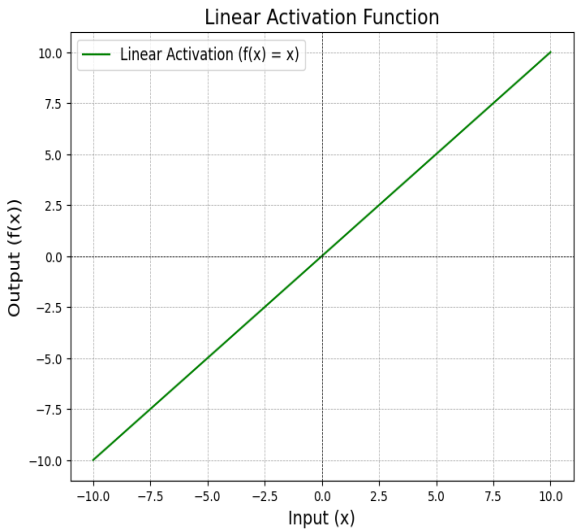

## 4. Hyperbolic Tangent (tanh) Function

- **Definition**: Similar to sigmoid but outputs values between -1 and 1.
- **Mathematical Representation**:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Limitations**: Also suffers from vanishing gradients but performs better than sigmoid in practice.

# Types of Activation Functions



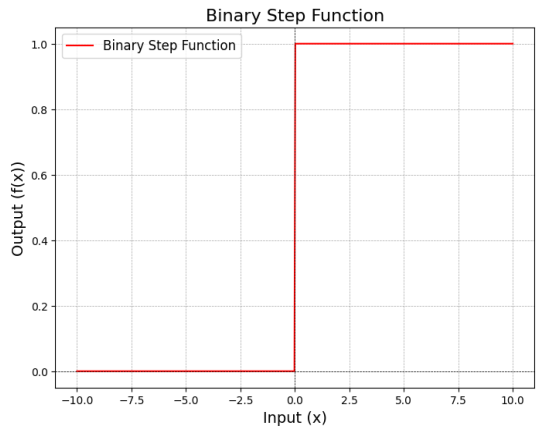

## 5. Rectified Linear Unit (ReLU)

- **Definition**: Outputs zero for negative inputs and linear for positive inputs.
- **Mathematical Representation**:

$$f(x) = \max(0, \mathrm{x})$$

- **Advantages**: Computationally efficient and helps mitigate vanishing gradient issues.
- **Limitations**: Can suffer from "dying ReLU" where neurons become inactive during training.

## 6. Leaky ReLU




- **Definition**: A variant of ReLU that allows a small gradient when the input is negative
- **Mathematical Representation**:

$$f(x) = \begin{cases} x & if\ x > 0 \\ \alpha x & if\ x \leq 0 \end{cases}$$

where $\alpha$ is a small constant (e.g., 0.01).

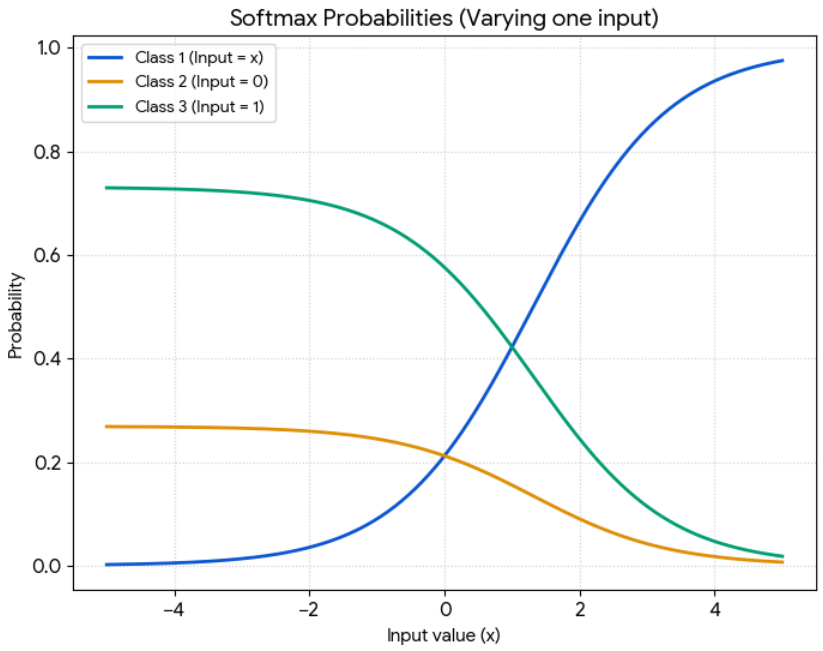
# Types of Activation Functions

## 7. Softmax Function

- **Definition**: Converts raw scores into probabilities for multi-class classification.
- **Mathematical Representation**:

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

for each class $i$.



# Choosing the Right Activation Function

- For binary classification tasks, sigmoid or binary step functions may be suitable.
- For multi-class classification tasks, softmax is typically used in the output layer.
- ReLU and its variants are often preferred in hidden layers due to their efficiency and effectiveness in training deep networks.